

Lab Experiment Sheet-1

School of Engineering and Technology

Course Code & Name: ENCS351 Operating System

Program Name: B.Tech CSE, AI ML, Data Science, Cyber, FSD, UX/UI

Name – Akshat Sharma

Course – BTECH CSE DS -Sem 5

Roll No - 2301420027

Summary of objectives

Task 1: Process Creation Utility

To accomplish this, I wrote a Python function called task1. I used a

for loop to call **os.fork()** three times, which created three distinct child processes. Inside the code block for each child (where

pid == 0), I used **os.getpid()** and **os.getppid()** to print its own Process ID and its parent's ID, along with a custom message. In the parent's code block, I made sure the parent process wouldn't exit prematurely by calling

os.waitpid() for each child, ensuring it waited for all of them to finish their execution.

Task 2: Command Execution Using **exec()**

For this task, I created a function task2 that forked a single child process. In the section of code executed by the child, I used

os.execvp("ls", ["ls", "-l"]). This system call replaced the child process's own code with the

ls -l command, effectively making the child execute that command in the terminal. The parent process simply waited for the command to finish before the script continued.

Task 3: Zombie & Orphan Processes

I simulated these two special process states in separate functions.

- **Zombie Process:** I created a child that printed a message and exited immediately using **os._exit(0)**. The key to creating a zombie was making the parent process

skip the `os.wait()` call. Instead, I made the parent sleep for 10 seconds. During this time, the child was "defunct" or a zombie because it had terminated, but the parent hadn't yet acknowledged its termination to clean it up from the process table.

- **Orphan Process:** I did the reverse for the orphan process. I made the

parent process exit immediately after forking, while the child process was programmed to sleep for 5 seconds. By the time the child woke up, its original parent was gone. I confirmed it had become an orphan by printing its new parent's PID (

`os.getppid()`), which had changed to 1 (the system's init process).

Task 4: Inspecting Process Info from `/proc`

I wrote a function

`inspect_process` that accepts a Process ID (PID) as an input. To get the required information, my script directly interacted with the

`/proc` virtual filesystem:

- I read and printed the

Name, State, and VmSize by opening and parsing lines from the `/proc/[pid]/status` file.

- I found the

executable's full path by using `os.readlink()` on the `/proc/[pid]/exe` symbolic link.

- I listed all

open file descriptors by using `os.listdir()` on the `/proc/[pid]/fd` directory.

Task 5: Process Prioritization

To demonstrate the effect of priority, my `task5` function forked multiple child processes. Inside each child, I assigned a different priority using the

`os.nice()` call, with values of 0, 5, and 10. A lower

`nice` value corresponds to a higher priority. After setting the priority, each child performed an identical, CPU-intensive calculation (a large summation loop). By observing the output, I confirmed that the child with the highest priority (the lowest

`nice` value) consistently finished its task first, showing the scheduler was giving it more CPU time

Code snippets

The screenshot shows a Kali Linux desktop environment. On the left, a file manager window displays the contents of the `os_lab1` directory, which includes a file named `process_management.py`. In the center, a terminal window shows the output of a command, displaying a table of process information with columns for PID, PPID, state, and other details. On the right, a nano editor window is open, showing the code for `process_management.py`. The code includes comments and function definitions for creating child processes, waiting for them to finish, and simulating zombie and orphan processes.

```
GNU nano 8.6 process_management.py
import os
import time
import sys

# Task 1: Create N child processes [cite: 35]
def task1(n=3):
    """Creates n child processes, each printing its PID and parent PID."""
    print("— Running Task 1: Process Creation —")
    pids = []
    for i in range(n):
        pid = os.fork()
        if pid == 0: # This is the child process
            print(f"Child (PID: {os.getpid()}) created by Parent (PPID: {os.getpid()}).")
            os._exit(0) # Child exits after printing
        else: # This is the parent process
            pids.append(pid)

    # Parent waits for all children to finish [cite: 36]
    for pid in pids:
        os.waitpid(pid, 0)
    print("— Task 1 Finished: Parent has waited for all children. —\n")

# Task 2: Execute a command using execvp [cite: 38]
def task2():
    """Creates a child process that executes a Linux command."""
    print("— Running Task 2: Command Execution —")
    pid = os.fork()
    if pid == 0: # Child process
        print(f"Child (PID: {os.getpid()}) is executing the 'ls -l' command.")
        try:
            os.execvp("ls", ["ls", "-l"])
        except FileNotFoundError:
            print("Error: Command not found.")
            os._exit(1)
    else: # Parent process
        os.wait()
        print("— Task 2 Finished: Child has executed the command. —\n")

# Task 3: Simulate Zombie and Orphan Processes [cite: 40]
def zombie_process():
    """Creates a zombie process."""
    print("— Running Task 3a: Zombie Process Simulation —")
    pid = os.fork()
    if pid == 0: # Child
        print(f"Zombie Child (PID: {os.getpid()}) created. It will exit immediately.")
        os._exit(0)
    else: # Parent
        print(f"Parent (PID: {os.getpid()}) is sleeping for 10 seconds, not calling os.wait().")
        time.sleep(10)
        print("Parent is done. The child should now be gone.")
        # We add a wait call here to clean up the zombie after demonstration
        os.wait()
    print("— Task 3a Finished. Check terminal output for 'ps' command. —\n")

def orphan_process():
    """Creates an orphan process."""
    print("— Running Task 3b: Orphan Process Simulation —")
    pid = os.fork()
    if pid == 0: # Child
        print(f"Child (PID: {os.getpid()}) is sleeping for 5 seconds.")
        time.sleep(5)
        # By the time the child wakes up, the original parent will be gone.
        # The child will be "adopted" by the 'init' process (PID 1).
        print(f"Child is awake. My PID is {os.getpid()}, and my Parent's PID is {os.getppid()}.")
        os._exit(0)
    else: # Parent
        print(f"Parent (PID: {os.getpid()}) is exiting immediately.")
        os._exit(0)

# Task 4: Inspect process information from /proc [cite: 43]
def inspect_process(pid):
    """Reads and prints details for a given PID from the /proc filesystem."""
    print(f"— Running Task 4: Inspecting PID {pid} —")
    try:
        # Read process name, state, and memory usage from /proc/[pid]/status
        with open(f"/proc/{pid}/status", "r") as f:
            for line in f:
                print(line)
    except:
        print(f"Error: PID {pid} does not exist.")
```

The screenshot shows a Kali Linux desktop environment. On the left, a file manager window displays the contents of the `os_lab1` directory, which includes a file named `process_management.py` and a screenshot of the terminal output. In the center, a terminal window shows the output of a command, displaying a table of process information with columns for PID, PPID, state, and other details. On the right, a nano editor window is open, showing the code for `process_management.py`. The code includes comments and function definitions for creating child processes, waiting for them to finish, and simulating zombie and orphan processes.

```
GNU nano 8.6 process_management.py
os.wait()
print("— Task 2 Finished: Child has executed the command. —\n")

# Task 3: Simulate Zombie and Orphan Processes [cite: 40]
def zombie_process():
    """Creates a zombie process."""
    print("— Running Task 3a: Zombie Process Simulation —")
    pid = os.fork()
    if pid == 0: # Child
        print(f"Zombie Child (PID: {os.getpid()}) created. It will exit immediately.")
        os._exit(0)
    else: # Parent
        print(f"Parent (PID: {os.getpid()}) is sleeping for 10 seconds, not calling os.wait().")
        time.sleep(10)
        print("Parent is done. The child should now be gone.")
        # We add a wait call here to clean up the zombie after demonstration
        os.wait()
    print("— Task 3a Finished. Check terminal output for 'ps' command. —\n")

def orphan_process():
    """Creates an orphan process."""
    print("— Running Task 3b: Orphan Process Simulation —")
    pid = os.fork()
    if pid == 0: # Child
        print(f"Child (PID: {os.getpid()}) is sleeping for 5 seconds.")
        time.sleep(5)
        # By the time the child wakes up, the original parent will be gone.
        # The child will be "adopted" by the 'init' process (PID 1).
        print(f"Child is awake. My PID is {os.getpid()}, and my Parent's PID is {os.getppid()}.")
        os._exit(0)
    else: # Parent
        print(f"Parent (PID: {os.getpid()}) is exiting immediately.")
        os._exit(0)

# Task 4: Inspect process information from /proc [cite: 43]
def inspect_process(pid):
    """Reads and prints details for a given PID from the /proc filesystem."""
    print(f"— Running Task 4: Inspecting PID {pid} —")
    try:
        # Read process name, state, and memory usage from /proc/[pid]/status
        with open(f"/proc/{pid}/status", "r") as f:
            for line in f:
                print(line)
    except:
        print(f"Error: PID {pid} does not exist.")
```




